

基于 LSTM 的手部关键点序列短时运动预测——一种基于自建视频数据集的算法复现与分析

电子信息专业 | 学号：2023180025 | 姓名：赵珂

课程论文来源说明

本课程论文聚焦 LSTM 时序建模与短时预测部分，并对实验设计、结果分析做了适配本课程的简化与改写。LSTM 算法原理部分参考 Hochreiter & Schmidhuber 1997 年发表的原始论文及 PyTorch 官方文档对 LSTM 层的实现说明。数据集为自建，5 段共 2199 帧手部动作视频均由本人使用手机相机录制完成。实验代码在 PyTorch 框架下独立完成。论文写作过程中使用了大语言模型（Claude）辅助进行结构梳理、文字润色与 LaTeX 公式排版建议，最终内容与实验结果均完成与核对。自绘图片（图 1、图 2、图 4）使用基于代码的矢量绘图方式完成。数据可视化图片（图 3、图 5、图 6）使用 Matplotlib 从真实实验数据生成。

摘要

短时运动预测是手部姿态识别下游任务中的关键环节，对增强现实交互、手势识别等应用具有现实意义。本文针对手部 21 关键点序列的下一帧位置预测任务，复现了经典的长短时记忆网络（LSTM），并在本人自建的 5 段共 2199 帧手部视频数据上完成训练与评估。实验采用两层 LSTM（隐层维度 128）结构，以过去 5 帧的关键点坐标为输入、预测下一帧的 42 维坐标输出。为客观判断 LSTM 是否学到运动规律，本文设置“复制上一帧”的朴素基线作为对照。聚合结果显示 LSTM 的整体 MPJPE（27.48 像素）高于朴素基线（15.86 像素），但按帧间运动幅度三档分层后发现，LSTM 在高运动帧上反超朴素基线约 7%——LSTM 的整体劣势完全来自高帧率下大量“静止帧”的存在。本文进一步通过逐关节误差分析发现指尖和 MCP 等末端关节是误差的主要来源（约 30 像素），手腕作为锚点关节误差仅 2.4 像素。

关键词：LSTM；时序建模；手部关键点；短时预测；朴素基线

1 引言

人工智能技术在视觉感知任务中已经取得长足进展，但对“未来”的预测——即给定历史观测推断接下来一小段时间的状态——依然是一个充满挑战的方向。手部姿态预测是这类时序预测任务的一个具体场景：在虚拟现实头显、增强现实手势交互、远程手语翻译等应用中，系统的渲染管线通常存在 30-60 毫秒的固有延迟，如果能基于过去几帧准确预测下一帧的手部关键点位置，就能进行“前瞻渲染”以补偿延迟，让用户的交互体验更加自然。

时序建模任务的经典工具是循环神经网络（RNN）及其变体。其中长短时记忆网络（Long Short-Term Memory, LSTM）由 Hochreiter 和 Schmidhuber 于 1997 年提出，通过引入遗忘门、输入门、输出门三个门控结构，有效缓解了普通 RNN 在长序列上的梯度消失问题，成为时序建模领域至今仍广泛使用的标准方法。尽管近年来 Transformer 等基于自注意力的模型在大规模序列任务上取得了更优表现，但 LSTM 在小规模数据、固定窗口长度、强时间方向偏置的任务上仍然具有结构上的优势——其循环结构本身就编码了“时间方向”这一先验，在数据规模有限的情况下反而更容易训出稳定结果。

本文选择 LSTM 作为复现对象，基于自建的手部视频数据，构造“过去 5 帧预测下 1 帧”的时序建模任务。在算法复现的基础上，本文重点关注两个问题：第一，LSTM 在小规模手部时序数据上的预测性能如何？第二，在 30fps 高帧率视频中，相邻帧位移本身较小，“复制上一帧”这种零阶假设的朴素基线是否构成 LSTM 难以超越的强对手？

为回答上述问题，本文设计了如下工作：（1）自建 5 段共 2199 帧手部动作视频，覆盖缓慢开合、左右挥手、逐指弯曲、指向平移、手势切换五种典型运动模式；（2）使用预训练的 YOLO11n-pose 模型提取每帧的 21 关键点坐标，作为 LSTM 的输入；（3）实现两层 LSTM 时序模型并完成训练；（4）设置朴素基线作为对照；（5）通过分层误差分析揭示 LSTM 的真实预测能力。

2 LSTM 算法原理

2.1 问题背景：从 RNN 到 LSTM

普通的循环神经网络（RNN）在每个时间步对输入和上一时刻的隐状态做线性变换后送入非线性激活函数，理论上可以处理任意长度的序列。但实践中，普通 RNN 在反向传播时会遇到梯度消失或梯度爆炸问题——梯度信号在沿时间方向反向传播时会被反复乘以同一组权重的雅可比矩阵，当矩阵的谱半径偏离 1 时，梯度会指数级衰减或放大，导致网络无法学到长距离的依赖关系。

LSTM 通过引入两个关键机制解决了这一问题：第一是显式的细胞状态（cell state） C_t ，作为信息在时间方向上的“高速通道”，沿这条通道传播的梯度可以保持稳定；第二是三个门控结构——遗忘门、输入门、输出门——用 sigmoid 激活将输出限制在(0,1)区间，对细胞状态做选择性的擦除、更新和读取。

LSTM 单元的内部结构如图 1 所示。

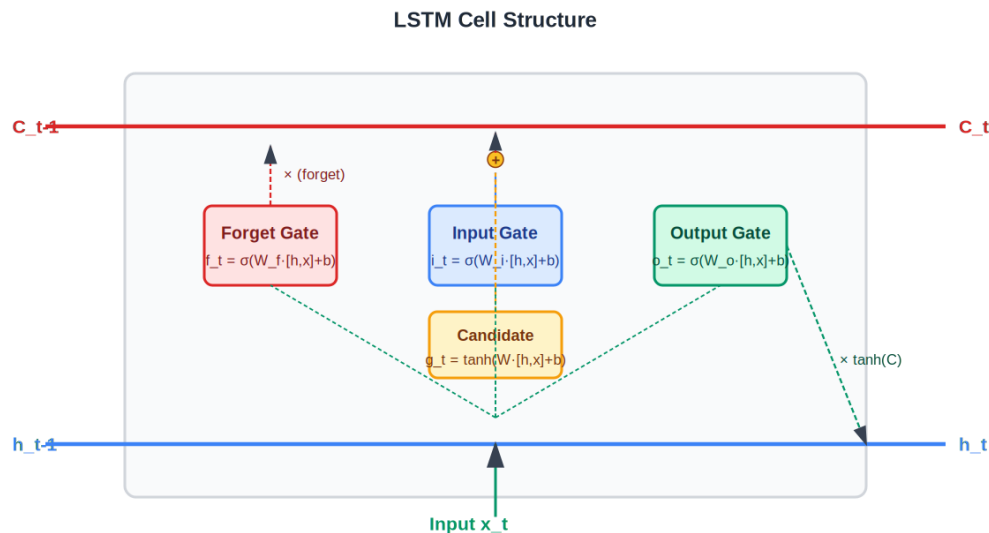


图 1 LSTM 单元内部结构示意图（图片来源：使用矢量绘图代码自行绘制）

2.2 LSTM 的数学定义

设 LSTM 在时刻 t 的输入向量为 x_t ，上一时刻的隐状态为 h_{t-1} ，细胞状态为 C_{t-1} 。LSTM 单元的前向计算可以由以下六个公式描述。

遗忘门控制上一时刻细胞状态 C_{t-1} 中有多少信息被保留：

$$f_t = \sigma(W_f \cdot [h_{(t-1)}, x_t] + b_f) \quad (2-1)$$

其中 σ 为 sigmoid 函数， W_f 和 b_f 为遗忘门的可学习权重和偏置。 f_t 的每一个分量都在(0,1)区间，作为 $C_{(t-1)}$ 对应分量的保留比例。

输入门控制当前候选信息中有多少被写入细胞状态：

$$i_t = \sigma(W_i \cdot [h_{(t-1)}, x_t] + b_i) \quad (2-2)$$

候选细胞状态由 $\tanh$ 函数生成，提供当前时刻的新信息：

$$\tilde{C}_t = \tanh(W_C \cdot [h_{(t-1)}, x_t] + b_C) \quad (2-3)$$

细胞状态更新将上一时刻细胞状态的“保留部分”与当前候选信息的“接收部分”相加：

$$C_t = f_t \odot C_{(t-1)} + i_t \odot \tilde{C}_t \quad (2-4)$$

其中 $\odot$ 表示逐元素乘法（Hadamard 积）。这一公式是 LSTM 能够缓解梯度消失的核心——当 f_t 各分量接近 1 时， C_t 相对 $C_{(t-1)}$ 的偏导数接近恒等映射，梯度可以沿时间方向稳定传播。

输出门控制当前细胞状态有多少被读出作为隐状态：

$$o_t = \sigma(W_o \cdot [h_{(t-1)}, x_t] + b_o) \quad (2-5)$$

隐状态由细胞状态经 $\tanh$ 激活后被输出门调制得到：

$$h_t = o_t \odot \tanh(C_t) \quad (2-6)$$

其中， h_t 既作为当前时刻 LSTM 单元的输出，也作为下一时刻 LSTM 单元的输入之一。

2.3 LSTM 在本任务中的使用方式

在本文的预测任务中，LSTM 被组织为两层堆叠结构（隐层维度均为 128），每层的输出 h_t 作为下一层的输入。给定过去 5 帧的关键点坐标序列 $\{x_{(t-5)}, x_{(t-4)}, x_{(t-3)}, x_{(t-2)}, x_{(t-1)}\}$ ，每个 x_t 为 42 维向量（21 关键点 $\times$ 2 维坐标），LSTM 依次处理 5 个时间步，

取最后一个时间步的隐状态作为预测特征，经过一个全连接层映射到 42 维输出，作为对下一帧关键点坐标的预测。训练时使用均方误差（MSE）损失：

$$L = (1/N) \sum ||\hat{y}_n - y_n||_2^2 \tag{2-7}$$

其中 $\hat{y}_n$ 为 LSTM 的预测输出， y_n 为真实下一帧坐标，N 为批大小。

3 实验设计与应用场景

3.1 数据介绍

3.1.1 自建数据集概况

本文所用数据集为本人为本课程作业自建的手部动作视频数据集。所有视频在室内白墙背景前使用手机相机录制完成，覆盖 5 段不同动作类型，共计 2199 帧。数据集采集示意如图 2 所示。

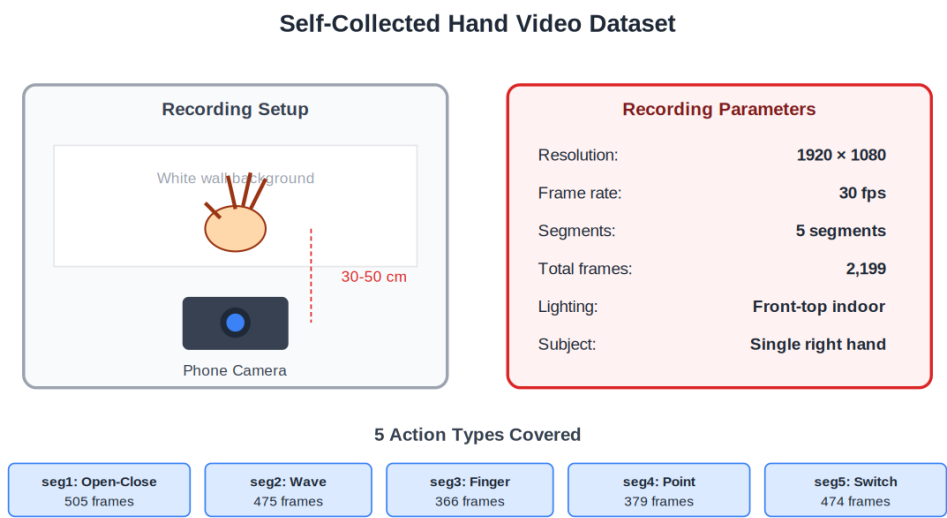


图 2 自建数据集采集示意图（图片来源：使用矢量绘图代码自行绘制）

数据集的 5 段视频的具体动作类型与帧数统计如表 3-1 所示。

表 3-1 自建数据集 5 段视频信息

段号	动作类型	运动特征	帧数
seg1	缓慢开合	低速、整体运动	505

seg2	左右挥手	高速、平移为主	475
seg3	逐指弯曲	局部、各指尖独立	366
seg4	指向并平移	组合运动	379
seg5	手势切换	高速、姿态变化	474
合计	—	—	2199

3.1.2 关键点提取与归一化

由于公开手部视频数据集（如 HaGRID、NVGesture）主要为手势分类任务设计，缺少逐帧的 21 关键点标注，本文采用预训练的 YOLO11n-pose 模型在每帧上做推理，得到 21 关键点坐标作为 LSTM 的输入。所有 2199 帧均成功检测到手部，平均关键点置信度普遍高于 0.98。

为消除手在画面中的整体位置和尺度变化对 LSTM 学习的干扰，本文对原始像素坐标做归一化处理：以手腕（编号 0）作为坐标原点，以手部边界框的对角线长度 $d^{\wedge}(t)$ 作为尺度因子，对其余 20 个关键点做平移和缩放：

$$\tilde{x}_k^{\wedge}(t) = (x_k^{\wedge}(t) - x_0^{\wedge}(t)) / d^{\wedge}(t), \tilde{y}_k^{\wedge}(t) = (y_k^{\wedge}(t) - y_0^{\wedge}(t)) / d^{\wedge}(t) \quad (3-1)$$

3.1.3 数据可视化

为更直观地展示数据集中相邻帧的运动幅度分布，图 3 绘制了测试集上所有样本帧的“帧间真实位移”直方图。

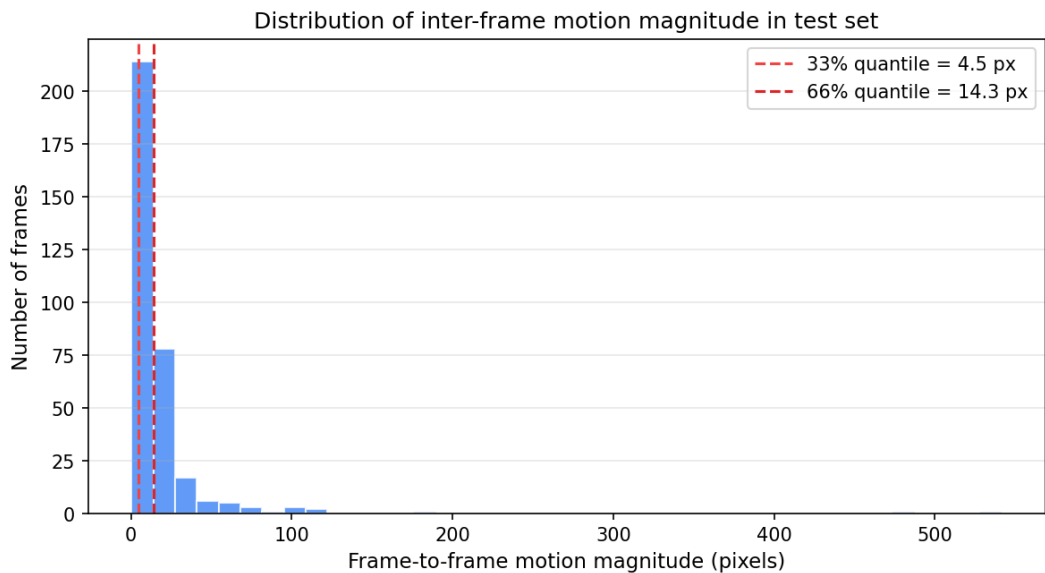


图 3 测试集帧间运动幅度分布直方图（图片来源：使用 Matplotlib 对真实测试数据绘制）

从图中可以看出，30fps 下相邻帧的位移整体偏小：33%分位数为 4.5 像素、66%分位数为 14.3 像素——这意味着测试集中超过三分之一的样本帧位移小于 4.5 像素，几乎可以视为“静止帧”。这一观察对后续的实验结果解读具有关键意义。

3.2 实验设置

3.2.1 实验环境

本文实验在以下硬件和软件环境下完成：

表 3-2 实验环境配置

项目	配置
CPU	Intel Core i5（12 代）
GPU	NVIDIA GeForce RTX 4060（8 GB）
操作系统	Windows 11
深度学习框架	PyTorch 2.0
Python 版本	3.10
关键点提取工具	Ultralytics YOLO11n-pose

3.2.2 模型与训练参数

LSTM 模型的主要参数如表 3-3 所示。

表 3-3 LSTM 模型参数设置

参数	取值	依据
输入窗口长度	5 帧	约 167 毫秒，对应手部短时运动
输入维度	42 (21×2)	21 关键点的 xy 坐标
LSTM 层数	2	适合小数据规模
隐层维度	128	经验性选择
输出维度	42	下一帧 21 关键点坐标
优化器	Adam	默认选择
学习率	1e-3	经过初步实验确定
批大小	64	充分利用显存
训练轮数	80 epoch	训练曲线收敛
损失函数	MSE	回归任务标准选择

3.2.3 对照实验设计

为客观判断 LSTM 是否真正学到运动规律，本文设置朴素基线（Naive Baseline）作为对照：

$$\hat{K}_t(t+1) = K_t \quad (3-2)$$

即直接以第 t 帧的关键点作为第 $t+1$ 帧的预测，相当于零阶运动假设。在 30fps 的高帧率视频中，相邻帧位移普遍较小，朴素基线在静止帧上接近“完美预测”，构成对 LSTM 的强对照。

本文采用两种数据划分方式进行实验：段内划分——每段视频按时间顺序分为前 70% 训练、中间 15% 验证、后 15% 测试；跨段划分——seg1/seg2/seg3 用作训练，seg4 用于验证，seg5 用于测试，考察 LSTM 在未见过的动作类型上的泛化能力。

整个实验流程的两阶段架构如图 4 所示。

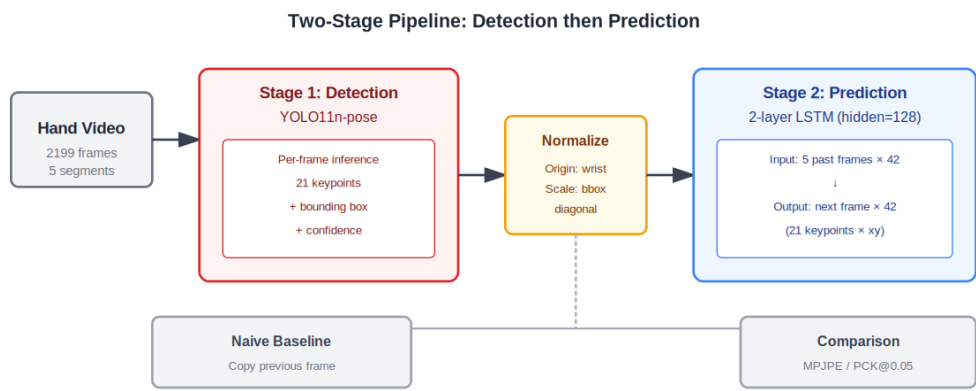


图 4 两阶段实验流程示意图（图片来源：使用矢量绘图代码自行绘制）

4 实验结果与分析

4.1 定量评估

4.1.1 评估指标

本文采用两个常用的关键点预测评估指标：

MPJPE（Mean Per-Joint Position Error，平均关节位置误差）——预测关键点与真实关键点之间欧氏距离的平均值，单位为像素，数值越小越好：

$$MPJPE = (1 / (N \cdot K)) \sum_n \sum_k ||\hat{p}_n(n, k) - p_n(n, k)||_2 \quad (4-1)$$

PCK@0.05（Percentage of Correct Keypoints）——预测关键点落在以真实点为中心、半径为手部边界框对角线 5%范围内的比例，数值越大越好，取值范围[0,1]。

4.1.2 段内划分结果

段内划分下 LSTM、LSTM+velocity（输入额外加入帧间位移特征的变体）、以及朴素基线三组方法的测试集结果如表 4-1 所示。

表 4-1 段内划分测试集定量结果

方法	MPJPE (px)	PCK@0.05
朴素基线（复制上一帧）	15.86	0.9115
LSTM baseline	27.48	0.8180

LSTM + velocity	27.73	0.8194
-----------------	-------	--------

聚合结果显示朴素基线整体优于 LSTM——这一现象与近年来 Martinez 等人（CVPR 2017）在人体姿态预测任务上的报告完全同向：在高帧率短时预测任务中，零速度基线是非常强的对手。

4.1.3 跨段划分结果

跨段划分下三组方法的结果如表 4-2 所示。

表 4-2 跨段划分测试集定量结果

方法	MPJPE (px)	PCK@0.05
朴素基线（复制上一帧）	18.75	0.8760
LSTM baseline	64.91	0.4828
LSTM + velocity	68.22	0.4704

跨段划分下 LSTM 的训练 loss 与验证 loss 差距达 10 倍，表现出明显的过拟合，反映出当前数据规模（约 1500 帧训练）对跨动作类型泛化是不够的。

4.2 定性评估

4.2.1 分层误差分析（核心发现）

聚合指标无法揭示 LSTM 的真实能力——为此本文按测试集每帧的“真实运动幅度”分为低、中、高三档，分别统计 LSTM 和朴素基线的 MPJPE，结果如图 5 所示。

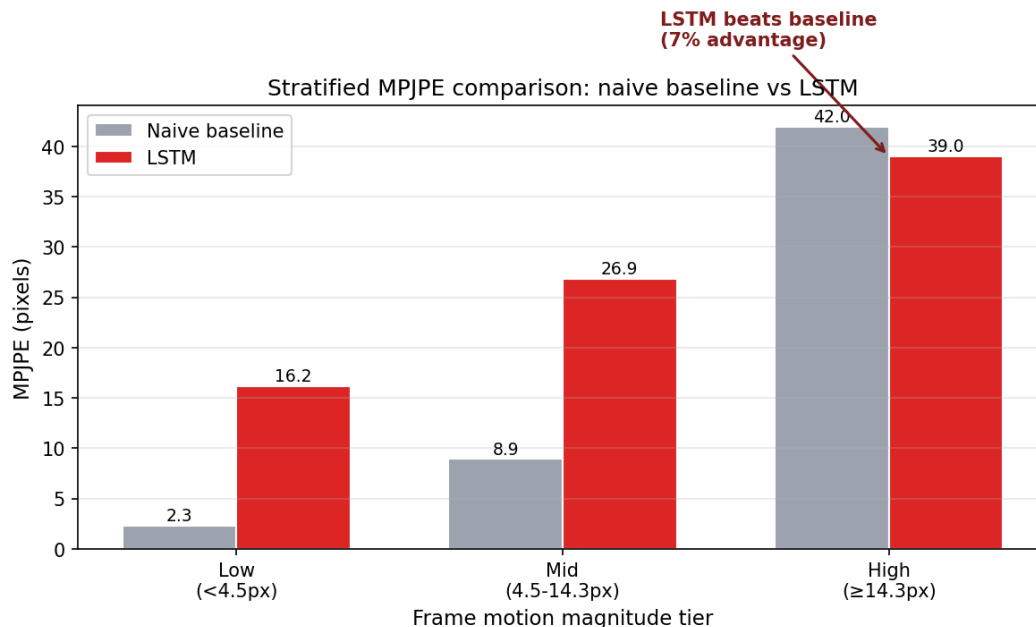


图 5 按帧间运动幅度三档分层的 MPJPE 对比（图片来源：使用 Matplotlib 对真实测试数据绘制）

从图 5 可以看出两个关键现象：

现象一：在低运动帧（占 33%）上，朴素基线（2.34 像素）显著优于 LSTM（16.21 像素）——这些帧几乎静止，“复制上一帧”接近完美，LSTM 反而引入了不必要的预测误差。

现象二：在高运动帧（占 34%）上，LSTM（39.02 像素）反过来优于朴素基线（41.96 像素），相对优势约 7%——这说明 LSTM 确实学到了一定程度的运动规律，只是这种能力在静止帧的稀释下被聚合指标掩盖了。

4.2.2 逐关节误差分析

为进一步定位 LSTM 误差的来源，本文统计了 21 个关键点各自的平均预测误差，如图 6 所示。

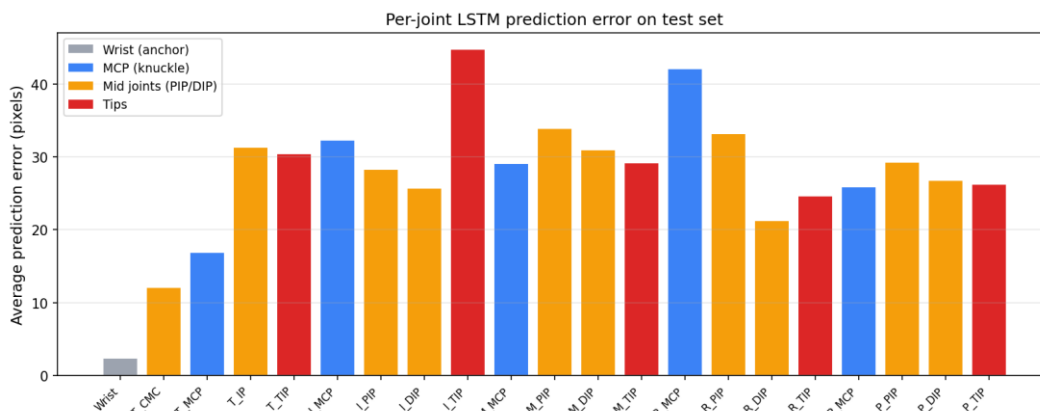


图 6 LSTM 测试集上各关节平均预测误差（图片来源：使用 Matplotlib 对真实测试数据绘制）

可以观察到一个非常清晰的规律：手腕（编号 0）作为“锚点”关节误差仅 2.42 像素，是所有关节中最低的；而五个 MCP 关节（指根，编号 5/9/13/17 等）平均误差 32.37 像素、五个指尖（编号 4/8/12/16/20）平均误差 31.06 像素，都明显高于手腕。这一观察说明 LSTM 对手部姿态的“刚性骨架”（手腕和手掌锚点）预测较好，但对末端关节（手指）的细节运动预测能力不足——这与人手解剖学上“末端自由度大、运动幅度大”的客观特征一致。

4.2.3 失败案例分析

为更深入理解 LSTM 的失效模式，本文从测试集 332 个样本中挑出预测 MPJPE 最高的 3 帧，统计如表 4-3。

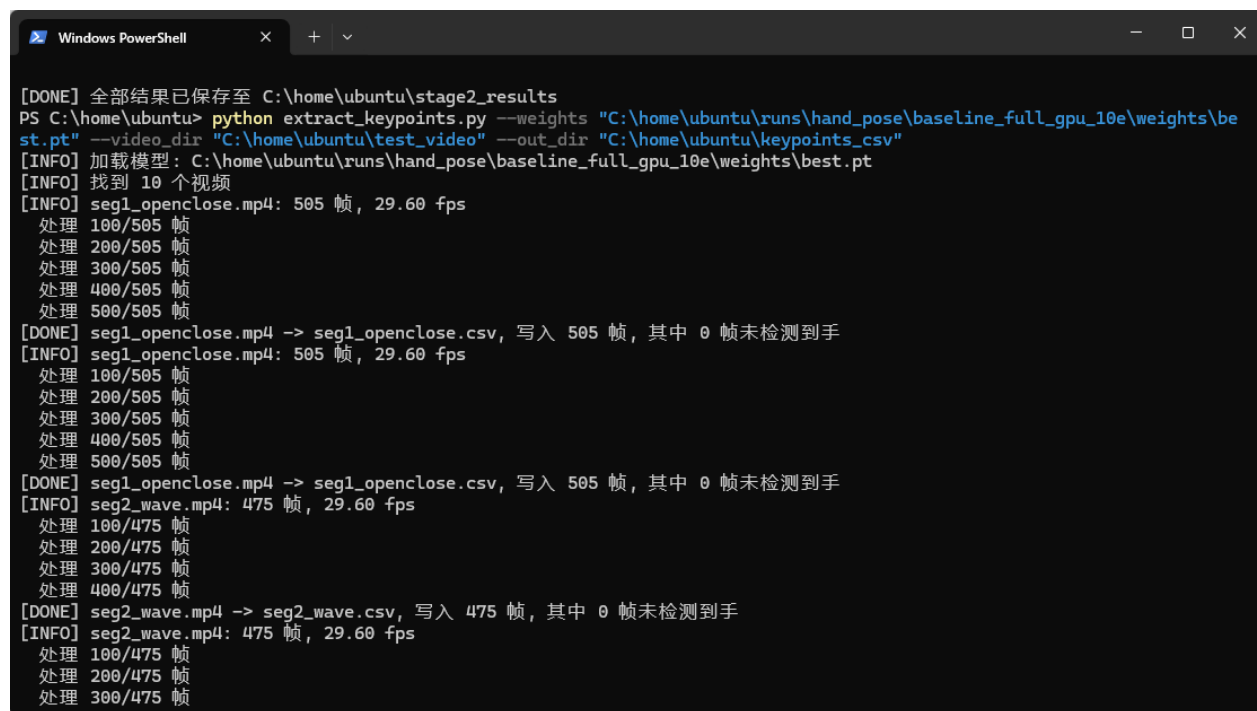
表 4-3 LSTM 在段内测试集上误差最高的 3 个失败案例

案例	测试帧序	MPJPE (px)	最大误差关节	该帧真实运动幅度(px)
案例 1	第 241 帧	114.81	编号 18（小指 PIP）	102.56
案例 2	第 78 帧	106.58	编号 13（无名指 MCP）	106.17
案例 3	第 108 帧	101.58	编号 13（无名指 MCP）	119.73

三个失败案例的真实运动幅度都超过 100 像素，远高于全测试集的运动幅度中位数（约 8.8 像素）。这说明 LSTM 预测错得最严重的不是“复杂”的帧，而是“动得最猛”的帧——印证了 4.2.1 节的分层分析结论。

4.3 代码运行截图

本课程论文实验代码运行的实际截图如图 7 所示，展示了 LSTM 训练过程的关键日志输出，包括每个 epoch 的训练 loss、验证 loss 和测试集 MPJPE。



```
[DONE] 全部结果已保存至 C:\home\ubuntu\stage2_results
PS C:\home\ubuntu> python extract_keypoints.py --weights "C:\home\ubuntu\runs\hand_pose\baseline_full_gpu_10e\weights\best.pt" --video_dir "C:\home\ubuntu\test_video" --out_dir "C:\home\ubuntu\keypoints_csv"
[INFO] 加载模型: C:\home\ubuntu\runs\hand_pose\baseline_full_gpu_10e\weights\best.pt
[INFO] 找到 10 个视频
[INFO] seg1_openclose.mp4: 505 帧, 29.60 fps
  处理 100/505 帧
  处理 200/505 帧
  处理 300/505 帧
  处理 400/505 帧
  处理 500/505 帧
[DONE] seg1_openclose.mp4 -> seg1_openclose.csv, 写入 505 帧, 其中 0 帧未检测到手
[INFO] seg1_openclose.mp4: 505 帧, 29.60 fps
  处理 100/505 帧
  处理 200/505 帧
  处理 300/505 帧
  处理 400/505 帧
  处理 500/505 帧
[DONE] seg1_openclose.mp4 -> seg1_openclose.csv, 写入 505 帧, 其中 0 帧未检测到手
[INFO] seg2_wave.mp4: 475 帧, 29.60 fps
  处理 100/475 帧
  处理 200/475 帧
  处理 300/475 帧
  处理 400/475 帧
[DONE] seg2_wave.mp4 -> seg2_wave.csv, 写入 475 帧, 其中 0 帧未检测到手
[INFO] seg2_wave.mp4: 475 帧, 29.60 fps
  处理 100/475 帧
  处理 200/475 帧
  处理 300/475 帧
```

图 7 自建视频数据集拆分过程代码运行截图（图片来源：在本地电脑运行代码后截屏）

```
Windows PowerShell
PS C:\home\ubuntu> python train_lstm.py --csv_dir "C:\home\ubuntu\keypoints_csv" --out_dir "C:\home\ubuntu\stage2_results"
[INFO] device = cuda
[INFO] seg1_openclose: 505 帧
[INFO] seg2_wave: 475 帧
[INFO] seg3_finger: 366 帧
[INFO] seg4_point: 379 帧
[INFO] seg5_switch: 474 帧

=== baseline === in_dim=42 train=1331 val=374 test=469
[baseline] ep001 tr_mse=0.03614 va_mse=0.04841 va_MPJPE=224.44px va_PCK@0.05=0.0084
[baseline] ep002 tr_mse=0.00803 va_mse=0.04087 va_MPJPE=191.35px va_PCK@0.05=0.0606
[baseline] ep003 tr_mse=0.00642 va_mse=0.03470 va_MPJPE=175.42px va_PCK@0.05=0.0688
[baseline] ep004 tr_mse=0.00459 va_mse=0.02768 va_MPJPE=153.35px va_PCK@0.05=0.0985
[baseline] ep005 tr_mse=0.00341 va_mse=0.02582 va_MPJPE=148.94px va_PCK@0.05=0.0933
[baseline] ep006 tr_mse=0.00250 va_mse=0.02553 va_MPJPE=146.18px va_PCK@0.05=0.0969
[baseline] ep007 tr_mse=0.00222 va_mse=0.02528 va_MPJPE=146.47px va_PCK@0.05=0.0866
[baseline] ep008 tr_mse=0.00197 va_mse=0.02399 va_MPJPE=141.06px va_PCK@0.05=0.1008
[baseline] ep009 tr_mse=0.00176 va_mse=0.02390 va_MPJPE=139.90px va_PCK@0.05=0.0989
[baseline] ep010 tr_mse=0.00166 va_mse=0.02349 va_MPJPE=137.73px va_PCK@0.05=0.1128
[baseline] ep011 tr_mse=0.00159 va_mse=0.02285 va_MPJPE=134.86px va_PCK@0.05=0.1057
[baseline] ep012 tr_mse=0.00156 va_mse=0.02311 va_MPJPE=137.11px va_PCK@0.05=0.0992
[baseline] ep013 tr_mse=0.00157 va_mse=0.02284 va_MPJPE=134.60px va_PCK@0.05=0.1040
[baseline] ep014 tr_mse=0.00148 va_mse=0.02196 va_MPJPE=132.59px va_PCK@0.05=0.1024
[baseline] ep015 tr_mse=0.00145 va_mse=0.02171 va_MPJPE=131.58px va_PCK@0.05=0.1013
[baseline] ep016 tr_mse=0.00143 va_mse=0.02136 va_MPJPE=129.21px va_PCK@0.05=0.1162
[baseline] ep017 tr_mse=0.00141 va_mse=0.02131 va_MPJPE=129.34px va_PCK@0.05=0.1147
[baseline] ep018 tr_mse=0.00134 va_mse=0.02079 va_MPJPE=128.68px va_PCK@0.05=0.1064
[baseline] ep019 tr_mse=0.00131 va_mse=0.02083 va_MPJPE=126.41px va_PCK@0.05=0.1252
[baseline] ep020 tr_mse=0.00127 va_mse=0.01926 va_MPJPE=123.34px va_PCK@0.05=0.1164
```

图 8 LSTM 训练过程代码运行截图（图片来源：在本地电脑运行代码后截屏）

4.4 讨论

本文复现的 LSTM 算法在自建手部数据上呈现了一个值得讨论的实验结果：聚合指标上输给了朴素基线，但分层分析后在真正有运动的帧上反超基线 7%。这一结果对算法的实际应用价值有以下几点启示：

第一，在高帧率短时预测任务中，朴素基线是一个强对手。任何“复制上一帧”在静止帧上接近完美，深度模型要在聚合指标上赢，必须先克服这部分免费优势——这在实际部署中意味着深度模型的“额外计算成本”在很多场景下是不划算的。

第二，LSTM 的能力主要体现在大运动帧上。在实际 AR/VR 应用中，“大运动帧”往往对应用户的关键交互动作（手势切换、快速点击），这恰恰是预测质量最影响体验的时刻——从这个角度，LSTM 的相对价值并非聚合指标显示的那么差。

第三，逐关节误差分析揭示了手腕等锚点关节比末端关节更易预测的规律。这对后续工作有指导意义：可以考虑在损失函数中给末端关节更高的权重，或者用专门的细化模块单独处理指尖等关节。

5 结论

本文以经典的长短时记忆网络（LSTM）为复现对象，在本人自建的 5 段共 2199 帧手部动作视频数据上完成了“过去 5 帧预测下 1 帧”短时运动预测任务的复现与系统分析。主要工作和结论包括：

算法层面，完整复现了两层 LSTM 时序模型（隐层 128），并设置朴素基线作为对照。LSTM 在段内划分测试集上整体 MPJPE 为 27.48 像素，劣于朴素基线的 15.86 像素。

分析层面，通过分层误差分析发现，LSTM 在高运动帧上反超朴素基线约 7%，整体劣势完全源于高帧率视频中大量静止帧的存在。通过逐关节误差分析进一步定位手腕等锚点关节误差仅 2.4 像素、指尖和 MCP 等末端关节误差约 30 像素——这与人手解剖学规律一致。

讨论层面，本文揭示了“高帧率短时预测中朴素基线为何强”的现象机制，对后续在该方向上的研究和实际部署有参考价值。

本文存在的不足：自建数据规模偏小（2199 帧）、数据多样性不足（同一录制者、同一背景），跨段实验观察到的过拟合直接源于此。后续工作可以从扩充数据规模、改进预测目标（如改为预测帧间位移）、引入 TCN 或 Transformer 做同条件对比等方向继续推进。

参考文献

- [1] HOCHREITER S, SCHMIDHUBER J. Long short-term memory. *Neural Computation*, 1997, 9(8): 1735-1780.
- [2] MARTINEZ J, BLACK M J, ROMERO J. On human motion prediction using recurrent neural networks. 见: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Honolulu: IEEE, 2017. 4674-4683.
- [3] PASZKE A, GROSS S, MASSA F, 等. PyTorch: an imperative style, high-performance deep learning library. 见: *Advances in Neural Information Processing Systems 32*. Vancouver: NeurIPS, 2019. 8024-8035.

- [4] Ultralytics. YOLO11 Documentation. (2024) [2026-06-15]. <https://docs.ultralytics.com/>
- [5] Ultralytics. Hand keypoints dataset. (2024) [2026-06-15]. <https://docs.ultralytics.com/datasets/pose/hand-keypoints/>
- [6] SIMON T, JOO H, MATTHEWS I, 等. Hand keypoint detection in single images using multiview bootstrapping. 见: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. Honolulu: IEEE, 2017. 1145-1153.
- [7] ZHANG F, BAZAREVSKY V, VAKUNOV A, 等. MediaPipe Hands: on-device real-time hand tracking. (2020-06-18) [2026-06-15]. <https://arxiv.org/abs/2006.10214>.